

Guía de Implementación - HU01: Registro de Cuenta Gratuita (Sandbox)

Esta guía detalla las consideraciones técnicas, de arquitectura y de diseño de base de datos que cada desarrollador debe tener en cuenta al implementar las tareas de la Historia de Usuario HU01.

El flujo de registro está diseñado en **dos pasos (2-step flow)** para prevenir el abuso de recursos y asegurar que solo usuarios con correos electrónicos válidos puedan crear entornos (Tenants) en el sistema. Es crucial mantener la **separación de responsabilidades (separation of concerns)** a lo largo de toda la implementación.

1. T07: Desarrollo Frontend (Angular)

El Frontend debe orquestar la experiencia de usuario dividiendo el proceso en el flujo de 2 pasos.

Consideraciones a tener en cuenta:

- **Flujo de 2 Pasos y Enrutamiento:**
 - La navegación debe iniciar en el `LandingPageComponent`, el cual servirá como punto de entrada y enrutará al usuario hacia el `RegisterComponent`.
 - El **Paso 1** (Init) se maneja en el `RegisterComponent`.
 - El **Paso 2** (Activación) se maneja en el `VerifyComponent`, al cual el usuario accederá mediante un enlace enviado a su correo electrónico.
 - **Validaciones Asíncronas:** Implementar validadores asíncronos en el `RegisterComponent` (`checkEmailAsync()`) para asegurar en tiempo real que el correo electrónico ingresado no esté ya registrado en el sistema.
 - **Mapeo de DTOs:**
 - **Paso 1:** Los datos capturados en `RegisterComponent` (nombres, email, password) deben mapearse estrictamente a la interfaz `InitRegistrationDTO_TS` antes de enviarlos a través del `AuthService.initRegistration()`.
 - **Paso 2:** Los datos capturados en `VerifyComponent` (el token recibido por URL y el companyName) deben mapearse a `ActivationDTO_TS` y enviarse mediante `AuthService.activateTenant()`.
-

2. T04: Desarrollo Backend - Core de Seguridad (Spring Boot)

El Core maneja el Paso 1 (Init) del registro, centrándose en la validación y la creación segura del usuario inicial de forma inactiva.

Consideraciones a tener en cuenta:

- **Creación del Registro en `users`** : Al recibir el `InitRegistrationDTO` en el `AuthController`, el `RegistrationService` debe crear el usuario en la tabla `users` con el campo `is_active = false`. Esto impide el acceso al sistema hasta que se valide el correo.
- **Seguridad de Contraseñas**: Antes de guardar al usuario, la contraseña debe ser cifrada obligatoriamente utilizando `BCryptPasswordEncoder`.
- **Generación de Token**: Se debe generar un token seguro y guardarlo en la tabla `verification_tokens` (asociado al `user_id` recién creado). Este token se usará posteriormente para la verificación.
- **Manejo de Excepciones**: Si el correo ya existe, se debe lanzar una `EmailAlreadyExistsException`. El `GlobalExceptionHandler` debe capturarla y retornar una respuesta HTTP con el status adecuado para que el Frontend la procese.

3. T05: Desarrollo Backend - Mensajería

Este módulo actúa como puente entre el Paso 1 y el Paso 2, encargado de notificar al usuario.

Consideraciones a tener en cuenta:

- **Uso del Token Generado**: Integrar con el `RegistrationService` para que, una vez guardado el `verification_token`, se llame al `EmailNotificationService`.
- **Envío de Correo**: Enviar un correo al usuario que contenga la URL hacia el `VerifyComponent` del Frontend, incluyendo el token como parámetro.
- **Expiración del Token (`expiry_date`)**: Al crear el token en el paso anterior, asegurarse de establecer un `expiry_date` razonable (ej. 24 horas). El servicio de mensajería y activación deben estar alineados con estas políticas de seguridad, y cualquier intento de validación posterior debe comprobar que la fecha actual no supere el `expiry_date`.

4. T06: Desarrollo Backend - Activación

Este módulo maneja el Paso 2 (Activación), completando el proceso de registro y aprovisionando los recursos finales.

Consideraciones a tener en cuenta:

- **Validación del Token:** El endpoint `activateTenant(ActivationDTO)` debe buscar el token en la tabla `verification_tokens`. Debe comprobar que exista, pertenezca a un usuario, y que el `expiry_date` no haya pasado.
 - **Actualización de Usuario:** Si el token es válido, se debe actualizar el registro del usuario en la tabla `users`, cambiando `is_active = true`.
 - **Creación del Tenant:** Aprovisionar el Sandbox creando un nuevo registro en la tabla `tenants` con el `name` provisto en el DTO y estableciendo `plan_type = 'SANDBOX'`.
 - **Asignación de Roles:** Enlazar al usuario verificado y al tenant creado insertando un registro en la tabla asociativa `user_tenant_roles` con `role = 'ADMINISTRADOR'`.
 - **Transaccionalidad (`@Transactional`):** Es **crítico** que el método orquestador (en `RegistrationService`) esté anotado con `@Transactional`. Esto asegura que si falla la creación del Tenant o la asignación del rol, no se marque al usuario como activo ni se consuma el token erróneamente, manteniendo la consistencia de la base de datos (atomicidad).
-

Resumen de Impacto en Base de Datos

- **Durante Init (Paso 1 - T04/T05):**
 - `users`: `INSERT` (con `is_active = false`).
 - `verification_tokens`: `INSERT` (con token y `expiry_date`).
 - `tenants`: *Sin cambios.*
 - `user_tenant_roles`: *Sin cambios.*
- **Durante Activación (Paso 2 - T06):**
 - `users`: `UPDATE` (cambia `is_active = true`).
 - `verification_tokens`: Posible `DELETE` o `UPDATE` para invalidar/marcar como usado el token.
 - `tenants`: `INSERT` (creación del sandbox con `plan_type = 'SANDBOX'`).
 - `user_tenant_roles`: `INSERT` (asignación de rol `ADMINISTRADOR` al usuario para su nuevo tenant).